МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

# Отчет по лабораторной работе №4

Специальность ПО-13

Выполнил:

Д. В. Шибун

студент группы ПО-13

Проверил:

А. Д. Кулик

13.02.2026

Брест 2026

## Цель работы:

научиться работать с Github API, приобрести практические навыки написания программ для работы с REST API или GraphQL API

## Общее задание:

Используя Github API, реализовать предложенное задание на языке Python. Выполнить визуализацию результатов, с использованием графика или отчета. Можно использовать как REST API (рекомендуется), так и GraphQL

## Задание: Анализ активности разработчиков в open-source проекте

## Условие:

Напишите Python-скрипт, который анализирует активность разработчиков в указанном публичном GitHub-репозитории и составляет рейтинг самых активных контрибьюторов.

1. Запрашивает у пользователя имя репозитория (например, pallets/flask или microsoft/vscode).
2. Использует GitHub API для получения списка всех контрибьюторов.
3. Для каждого контрибьютора собирает данные:
   - Количество коммитов
   - Количество открытых pull requests
   - Количество закрытых pull requests
   - Количество открытых issues
   - Количество закрытых issues
   - Дата последней активности
4. Определяет самых активных разработчиков по суммарному вкладу.
5. Строит график вклада разработчиков (matplotlib / seaborn).

Введите репозиторий для анализа (owner/repo): fastapi/fastapi

Анализируем вклад контрибьюторов в "fastapi/fastapi"...

ТОП-5 самых активных разработчиков:

1. user1 - 150 коммитов, 40 PR, 20 issues
2. user2 - 120 коммитов, 35 PR, 15 issues
3. user3 - 110 коммитов, 25 PR, 10 issues
4. user4 - 90 коммитов, 20 PR, 8 issues
5. user5 - 80 коммитов, 15 PR, 5 issues

Графики активности сохранены в "fastapi_contributors.png"

**Код программы:**

```
import requests
import matplotlib.pyplot as plt
import datetime
import os

#GitHub API helper
def github_get(url, params=None):
    headers = {"Accept": "application/vnd.github+json"}
    token = os.getenv("GITHUB_TOKEN")
```

```
    if token:
        headers["Authorization"] = f"Bearer {token}"

    response = requests.get(url, headers=headers, params=params)
    response.raise_for_status()
    return response.json()

#Collect contributor stats
def get_contributors(owner, repo):
    url = f"https://api.github.com/repos/{owner}/{repo}/contributors"
    return github_get(url)


def get_pull_requests(owner, repo, state):
    url = f"https://api.github.com/repos/{owner}/{repo}/pulls"
    return github_get(url, params={"state": state, "per_page": 100})


def get_issues(owner, repo, state):
    url = f"https://api.github.com/repos/{owner}/{repo}/issues"
    return github_get(url, params={"state": state, "per_page": 100})


def get_user_last_activity(username):
    url = f"https://api.github.com/users/{username}/events/public"
    events = github_get(url)
    if not events:
        return None
    return events[0]["created_at"]

#Main analysis function
def analyze_repo(full_name):
    owner, repo = full_name.split("/")
    print(f'Анализируем вклад контрибьюторов в "{full_name}"...')

    contributors = get_contributors(owner, repo)

    open_prs = get_pull_requests(owner, repo, "open")
    closed_prs = get_pull_requests(owner, repo, "closed")

    open_issues = get_issues(owner, repo, "open")
    closed_issues = get_issues(owner, repo, "closed")

    stats = []

    for c in contributors:
        username = c["login"]
        commits = c.get("contributions", 0)

        prs_open = len([p for p in open_prs if p["user"]["login"] == username])
        prs_closed = len([p for p in closed_prs if p["user"]["login"] == username])

        issues_open = len([i for i in open_issues if i["user"]["login"] == username])
        issues_closed = len([i for i in closed_issues if i["user"]["login"] == username])

        last_activity = get_user_last_activity(username)
```

```
        total_score = commits + prs_open + prs_closed + issues_open + issues_closed

        stats.append({
            "user": username,
            "commits": commits,
            "prs": prs_open + prs_closed,
            "issues": issues_open + issues_closed,
            "last_activity": last_activity,
            "score": total_score
        })

    # сортировка по активности
    stats_sorted = sorted(stats, key=lambda x: x["score"], reverse=True)

    return stats_sorted


#Plotting
def plot_stats(stats, repo_name):
    top = stats[:10]

    users = [s["user"] for s in top]
    scores = [s["score"] for s in top]

    plt.figure(figsize=(12, 6))
    plt.bar(users, scores, color="skyblue")
    plt.title(f"Активность контрибьюторов в {repo_name}")
    plt.ylabel("Суммарный вклад")
    plt.xticks(rotation=45, ha="right")

    filename = repo_name.replace("/", "_") + "_contributors.png"
    plt.tight_layout()
    plt.savefig(filename)
    print(f'График сохранён в "{filename}"')


#Entry point
if __name__ == "__main__":
    repo = input("Введите репозиторий (owner/repo): ").strip()
    stats = analyze_repo(repo)

    print("\nТОП-5 самых активных разработчиков:")
    for i, s in enumerate(stats[:5], start=1):
        print(f"{i}. {s['user']} — {s['commits']} коммитов, {s['prs']} PR, {s['issues']} issues")

    plot_stats(stats, repo)
```

**Спецификация ввода**

Введите репозиторий (owner/repo): <строка>

**Пример**

Введите репозиторий (owner/repo): kroschenko/spp_po13

**Спецификация вывода**
Анализируем вклад контрибьюторов в "<репозиторий>"...
ТОП-5 самых активных разработчиков:
1. <user> — <коммиты> коммитов, <PR> PR, <issues> issues
2. <user> — <коммиты> коммитов, <PR> PR, <issues> issues
3. <user> — <коммиты> коммитов, <PR> PR, <issues> issues
4. <user> — <коммиты> коммитов, <PR> PR, <issues> issues
5. <user> — <коммиты> коммитов, <PR> PR, <issues> issues
График сохранён в "<имя_файла>.png"

**Пример**

Введите репозиторий (owner/repo): kroschenko/spp_po13
Анализируем вклад контрибьюторов в "kroschenko/spp_po13"...

ТОП-5 самых активных разработчиков:
1. carrotist-coder — 35 коммитов, 5 PR, 5 issues
2. kroschenko — 40 коммитов, 1 PR, 1 issues
3. skumbriya21 — 32 коммитов, 5 PR, 5 issues
4. Katapo13 — 33 коммитов, 4 PR, 4 issues
5. kwappeta — 19 коммитов, 8 PR, 8 issues

График сохранён в "kroschenko_spp_po13_contributors.png"

**Рисунки с результатами работы программы**

```
Введите репозиторий (owner/repo): kroschenko/spp_po13
Анализируем вклад контрибьюторов в "kroschenko/spp_po13"...

ТОП-5 самых активных разработчиков:
1. carrotist-coder — 35 коммитов, 5 PR, 5 issues
2. kroschenko — 40 коммитов, 1 PR, 1 issues
3. skumbriya21 — 32 коммитов, 5 PR, 5 issues
4. Katapo13 — 33 коммитов, 4 PR, 4 issues
5. kwappeta — 19 коммитов, 8 PR, 8 issues
График сохранён в "kroschenko_spp_po13_contributors.png"
```

**Вывод:** В ходе выполнения работы был разработан Python-скрипт, использующий GitHub API для анализа активности разработчиков в выбранном публичном репозитории. Программа автоматически собирает данные о количестве коммитов, pull-request’ов, issues и дате последней активности каждого контрибьютора, после чего формирует рейтинг наиболее активных участников проекта.